

# Methods

## 1) Predefined Methods (Built-in Methods)

These are methods that are **already provided by Java libraries**. You don't need to write them—you just use them.

They are part of classes from packages like:

- java.lang
- java.util
- java.io

### ✓ Examples

```
public class Test {  
    public static void main(String[] args) {  
        String name = "iqbal";  
  
        // Predefined method  
        System.out.println(name.toUpperCase()); // IQBAL  
  
        System.out.println(Math.sqrt(16)); // 4.0  
    }  
}
```

### Explanation

- toUpperCase() → comes from String class
- sqrt() → comes from Math class
- println() → comes from PrintStream class

👉 These methods are already implemented by Java.

## ✓ Key Points

- No need to define logic
- Just call and use
- Saves development time
- Highly optimized and tested

```
public static void main(String[] args)
```

this is also a pre defined method

## 1) public

- **Access modifier**
- Means this method is **accessible from anywhere**

👉 JVM (Java Virtual Machine) must call this method from outside the class, so it must be public.

## 2) static

- Belongs to the **class**, not an object

👉 JVM can call main() **without creating an object** of the class.

If it was not static:

```
Test obj = new Test();
```

```
obj.main(); // JVM would need this, which it doesn't do
```

## 3) void

- **Return type**
- Means this method **does not return anything**

👉 JVM doesn't expect any value back after execution.

## 4) main

- **Method name**
- This is the **entry point of the program**

👉 JVM specifically looks for a method named main.  
If you change the name → program won't run.

## 5) String[] args

This part is slightly tricky 👉

### ➤ String[]

- Array of Strings
- Used to store **command-line arguments**

### ➤ args

- Variable name (you can change it, e.g., abc)

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
    }  
}
```

run with this `java Test Hello`

- public → must be public
- static → must be static
- void → must be void
- main → exact name required
- String[] args → parameter is required (name can change)

## 2) User-Defined Methods

These are methods that **you create based on your requirement**.

### ✅ Example

```
public class Test {  
  
    // User-defined method  
    public static void greet() {  
        System.out.println("Hello Iqbal!");  
    }  
  
    public static void main(String[] args) {  
        greet(); // calling method  
    }  
}
```

### Explanation

- greet() is created by you
- It contains custom logic

## Below are the types of user defined method

### Method without argument & without return type

👉 Takes **no input** and **does not return anything**

👉 Used when you just want to **perform an action**

```
class Test {  
  
    void greet() {  
        System.out.println("Hello!");  
    }  
  
    public static void main(String[] args) {
```

```
    Test obj = new Test();
    obj.greet();
}
}
```

## 2) Method without argument & with return type

👉 Takes **no input** but **returns a value**

👉 Used when result is **fixed or internally calculated**

```
class Test {

    int getNumber() {
        return 10;
    }

    public static void main(String[] args) {
        Test obj = new Test();
        System.out.println(obj.getNumber());
    }
}
```

## 3) Method with argument & without return type

👉 Takes **input from user** but **does not return anything**

👉 Used when you **process input and directly print/use it**

```
class Test {

    void display(int x) {
        System.out.println("Value: " + x);
    }

    public static void main(String[] args) {
        Test obj = new Test();
    }
}
```

```
    obj.display(5);  
}  
}
```

## 4) Method with argument & with return type

👉 Takes **input** and **returns result**

👉 Most commonly used type (real-world logic)

```
class Test {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        int result = obj.add(2, 3);  
        System.out.println(result);  
    }  
}
```

### Quick Summary

| Type | Argument | Return | Use Case                    |
|------|----------|--------|-----------------------------|
| 1    | ✗        | ✗      | Just perform action         |
| 2    | ✗        | ✓      | Return fixed/internal value |
| 3    | ✓        | ✗      | Take input, print/process   |
| 4    | ✓        | ✓      | Take input, return result   |